DS 2 Informatique - 9 octobre : 4h

Le sujet est décomposé en 4 parties. Vous devez choisir une partie à traiter entre la partie 3 et la partie 4. Votre choix doit être clairement indiqué.

1 Préliminaires

On rappelle quelques notions de graphes :

- Un ensemble I de sommets d'un graphe $G=(S,A)$ est dit indépendant s'il vérifie :

$$\forall (s,t)\in I^2, (s,t)\notin A.$$

- Un ensemble indépendant I est dit maximal pour un graphe G si on ne peut pas ajouter de sommets supplémentaires à I en conservant la notion d'indépendance.
- Un ensemble indépendant I est dit maximum pour un graphe G s'il s'agit d'un plus grand ensemble indépendant de G pour la cardinalité.
- Le graphe complémentaire d'un graphe G est un graphe G' ayant les mêmes sommets et tels que deux sommets distincts de G' soient adjacents si et seulement si il ne le sont pas dans G.
- Un ensemble C de sommets d'un graphe $G=(S,A)$ est une couverture par les sommets s'il vérifie :

$$\forall (s,t)\in A, s\in C \vee t\in C$$

Question 1 : Soit n un entier. Calculer la taille d'un indépendant maximum d'un graphe complet de taille n.

Correction :
Montrons qu'un indépendant I d'un graphe complet contient au plus un élément.

$$\forall (s,t)\in I^2, (s,t)\notin A(G) \Rightarrow s=t$$

Puisqu'un sommet seul est un ensemble indépendant, la taille d'un ensemble indépendant maximum d'un graphe complet de taille n est 1.

Question 2 : Calculer la taille d'un indépendant maximum d'un cycle de taille n.

Correction :
On distingue deux cas suivant la parité de n.

- Cas n pair
  Deux sommets consécutifs ne peuvent pas être dans un même ensemble indépendant. La taille d'un ensemble indépendant est donc borné par $n/2$. En numérotant les sommets de 1 à $2m$, l'ensemble $\{s_{2i} | \in [\![1,m]\!]\}$ est un ensemble indépendant. C'est donc bien la taille d'un indépendant maximum.
- Cas n impair
  La taille d'un indépendant maximum est $(n-1)/2$. $n/2$ n'étant pas entier, la borne du cas précédent est donc $(n-1)/2$. Il suffit donc de montrer qu'il existe bien un ensemble indépendant de cette taille. En numérotant les sommets de 1 à $2m+1$, l'ensemble $\{s_{2i} | \in [\![1,m]\!]\}$ est un ensemble indépendant ce qui conclut.

Question 3 : Prouver que S' est un ensemble indépendant de $G=(S,A)$ si et seulement si $S\setminus S'$ est une couverture par les sommets.

Correction :
Cette question est en fait fausse, seule la réciproque est vraie.

Dans la suite du sujet, on utilisera le type structuré suivant pour représenter une liste chaînée d'entiers :

```
struct Liste
{
    int val;
    struct Liste* suivant;
};
typedef struct Liste liste;
```

Un graphe G sera représenté par un tableau `G` de listes d'adjacence, c'est-à-dire un objet de type `liste**` de telle sorte que si $s \in S$, alors `G[s]` est la liste chaînée des voisins de s. On associera à ce tableau sa taille `n`.

```
struct Graphe
{
    int n;
    liste** voisins;
};
typedef struct Graphe graphe;
```

On représentera de plus un sous ensemble de S à l'aide d'un tableau de booléen de taille n. La case i du tableau vaudra `true` si et seulement si i appartient au sous-ensemble.

Question 4 : Écrire une fonction `bool* extraire_indep(bool* ensemble, int taille)` qui, étant donné une couverture par les sommets de S, renvoie l'ensemble indépendant associé. Le paramètre `taille` représente le nombre d'éléments de S.

Correction :
On alloue l'espace mémoire nécessaire au tableau puis l'on renvoie celui-ci une fois correctement rempli.

```
bool* extraire_indep(bool* ensemble, int taille)
{
    bool* res = (bool*) malloc(sizeof(bool)*taille);
    for (int i = 0; i < taille; i++)
    {
        res[i] = !ensemble[i];
    }
    return res;
}
```

Dans la suite du sujet, on étudiera des méthodes pour construire des ensembles indépendants de taille aussi grande que possible.

2 Algorithmes gloutons

On propose d'étudier un algorithme glouton pour trouver un ensemble indépendant de taille maximume. Celui-ci est décrit dans l'algorithme 1.

Question 5 : Démontrer que l'algorithme termine et que l'ensemble renvoyé est bien indépendant.

Correction :
L'algorithme termine puisque $|S|$ est un variant de la boucle principale. En effet, $|S|$ est entier, minoré par 0 et décroît strictement à chaque itération puisque l'on retire le sommet sélectionné s. La boucle `For` termine car chaque sommet a un nombre fini de voisins.
Pour montrer que l'ensemble renvoyé est bien indépendant, on remarque simplement que si un sommet est sélectionné ses voisins sont retirés de S. Ils ne pourront donc plus être sélectionnés. Si l'on souhaitais formaliser cela, on montrerait que la propriété : "I est un ensemble indépendant et aucun sommet présent dans I n'a de voisins dans S" est un invariant de la boucle principale.

Algorithme 1 Glouton(G)

$I \leftarrow \emptyset$
tant que $G \neq \emptyset$ **faire**
  $s \leftarrow$ un sommet de G.
  $I \leftarrow I \cup \{s\}$.
  pour t voisins de s **faire**
    Retirer t de S.
  fin pour
  Retirer s de S.
fin tant que
return I

Question 6 : Démontrer qu'il existe un graphe pour lequel l'algorithme ne renvoie pas une solution optimale.

Correction :
On considère un graphe à $n+1$ sommets. Les seules arêtes sont celles entre le sommet 1 et les autres sommets. Si l'algorithme commence par sélectionner le sommet 1, il obtiendra un ensemble indépendant de taille 1. Il est possible d'obtenir un ensemble indépendant de taille n en sélectionnant tous les autres sommets.

Dans la suite de la section, on suppose que l'on choisit s de degré minimal. On modifie le type de la section précédente de la façon suivante :

```
struct Graphe
{
    int n;
    liste** voisins;
    bool* tableau;
};

typedef struct Graphe graphe;
```

L'attribut `tableau` de taille n indique si un sommet est toujours présent dans le graphe. Ainsi, si s a été retiré précédemment, `tableau[s]` vaut `false`.

Question 7 : Écrire une fonction `int degre(graphe g, int s)` qui, étant donnés un graphe g et un sommet s, calcule le degré de s dans g. On fera particulièrement attention à ne pas compter les sommets déjà retirés.

Correction :
On parcours les voisins de s en augmentant le degré de 1 à chaque fois que l'on en trouve un qui n'a pas encore été retiré.

```
int degre(graphe g, int s)
{
    int d = 0;
    liste* voisins = g.tableau[s];
    while (voisins!=NULL)
    {
        if (g.tableau[voisins->val])
        {
            d=d+1;
        }
        voisins = voisins -> suivant;
    }
    return d
}
```

Question 8 : Écrire une fonction `int degre_minimal(graphe g)` qui, étant donné un graphe, calcule son sommet de degré minimal.

Correction :
On parcourt les sommets de g. Si un sommet n'a pas été retiré, on calcule son degré. et l'on met à jour les valeurs de `s_min` et `d_min` en fonction.

```
int degre_minimal(graphe g)
{
    int s_min = 0;
    int d_min = -1;
    for (int i = 0; i<g.n; i++)
    {
        if (g.tableau[i])
        {
            int d = degre(g,i);
            if (d_min =-1 || d<d_min)
            {
                d_min=d;
                s_min=i;
            }
        }
    }
    return s;
}
```

Question 9 : Écrire une fonction `bool* independant_glouton(graphe g)` qui, étant donné un graphe g, renvoie un ensemble indépendant obtenu par l'algorithme glouton.

Correction :
J'ai ici introduit deux fonctions :

- `retirer` qui, étant donné un graphe et un sommet de celui-ci, le retire du graphe ainsi que tous ses voisins.
- `est_vide` qui, étant donné un graphe, teste s'il ne possède aucun sommet.

```
bool est_vide(graphe g)
{
    for (int i = 0; i < g.n; i++)
    {
        if (g.tableau[i])
        {
            return false;
        }
    }
    return true;
}

void retirer(graphe g, int s)
{
    liste* voisins = g.voisins[s];
    while (liste!=NULL)
    {
        g.tableau[voisins->val] = false;
    }
    g.tableau[s]=false;
}

bool* independant_glouton(graphe g)
{
    bool* I = (bool*) malloc(sizeof(bool)*g.n);
    while (!est_vide(g))
    {
        int s = degre_minimal(g);
        I[s] = true;
        retirer(g,s);
    }
    return I;
}
```

Soit k un entier. On considère $G_k = ([\![0,2k]\!], A_k)$ le graphe défini de la façon suivante :

$$\forall i \in [\![1,k]\!],\ (0,i) \in A_k$$
$$\forall i \in [\![1,k]\!], \forall j \in [\![k+1,2k]\!]\ (i,j), \in A_k$$
$$\forall i,j \in [\![k+1,2k]\!],\ (i,j) \in A_k$$

Question 10 : Montrer que G_k admet un ensemble indépendant de taille k alors que l'algorithme glouton renvoie un ensemble de taille 2.

Correction :
Le sommet 0 est de degré k. Tous les sommets de $[\![1,k]\!]$ sont de degré $k+1$. Enfin, les autres sommets sont de degré $2k-1$. L'algorithme glouton commencera donc par choisir le sommet 0 et retirera alors tous les sommets de $[\![1,k]\!]$. Les sommets restants sont tous de degré $k-1$, et en choisissant un, le graphe devient alors vide. L'algorithme renvoie donc un ensemble de taille 2. Or, l'ensemble $[\![1,k]\!]$ est indépendant et de taille k.

Question 11 : En déduire que le rapport entre la taille de l'ensemble indépendant maximum et celle de l'ensemble indépendant renvoyé est non borné.

Correction :
L'analyse précédente est vraie pour tout entier k. Ainsi, pour tout graphe G_k, le rapport est de $k/2$ et n'est donc pas borné.

3 Graphes bipartis

On rappelle qu'un graphe G est dit biparti si l'existe une partition de ses sommets en deux ensembles telle qu'il n'y ait aucune arête entre deux sommets du même ensemble. Dans cette partie, on restreint notre étude aux graphes G bipartis . On note X et Y la partition des sommets qui en résulte.

3.1 Théorème de König

On cherchera tout d'abord à prouver le résultat suivant, connu sous le nom de théorème de König :

> Pour tout graphe biparti G, le cardinal maximal d'un couplage de G est égal au cardinal minimal d'une couverture par les sommets de G.

Question 12 : Prouver que, dans tout graphe, le cardinal de tout couplage est inférieur ou égal au cardinal de toute couverture par les sommets.

Correction :
Soit C un couplage de G et E une couverture par les sommets. Pour chaque arête dans le couplage, l'une des deux extrémités appartient à E puisqu'il s'agit d'une couverture. Donc, $|C| \leq |E|$.

Question 13 : En déduire que, pour tout graphe G, le cardinal maximal d'un couplage est inférieur ou égal au cardinal minimal d'une couverture par les sommets.

Correction :
La propriété de la question précédente est vraie pour le couplage de cardinal maximal C_0 et la couverture de cardinal minimal. On en déduit la propriété recherchée.

Question 14 : Rappeler la définition de chemin anternant associé à un couplage dans un graphe biparti.

Correction :
Un chemin alternant est un chemin du graphe dont qui vérifie la propriété suivante : parmi deux arêtes consécutives du chemin, exactement l'une d'entre elle appartient au couplage.

Étant donné un couplage C de taille maximale, on note S_0 l'ensemble des sommets de X qui ne sont pas extrémité d'une arête de C. De plus, on note S_1 l'ensemble des sommets de G qui sont soit dans S_0, soit accessibles depuis un sommet de S_0 en suivant un chemin alternant dans G.

Question 15 : Proposer un couplage de taille maximale du graphe de la figure 1. Justifier du caractère maximum de celui-ci et caractériser les valeurs de S_0 et S_1 associées.

Correction :
On peut choisir le couplage $\{(0,5),(2,6),(3,8),(1,7)\}$. Il est maximum car la taille d'un couplage est borné par le fait que 4 et 1 ne peuvent être tout incidents à un même couplage.
$S_0 = \{1\}$
$S_1 = \{1,7,4\}$

Question 16 : Montrer que toute arête de C admet soit ses deux extrémités dans S_1, soit aucune.

Correction :
Soit (s,t) une arête de C. Si s appartient à S_1, il a été ajouté car il est accessible à l'aide d'un chemin c alternant depuis un sommet de S_0. Soit t' le sommet avant s dans le chemin.

- Si $t = t'$, on a bien t qui appartient à S_1 car le chemin c tronqué à t est toujours un chemin alternant depuis un sommet de S_0.
- Sinon, (s,t') n'est pas une arête du couplage car (s,t) en est déjà une. Donc le chemin c prolongé avec

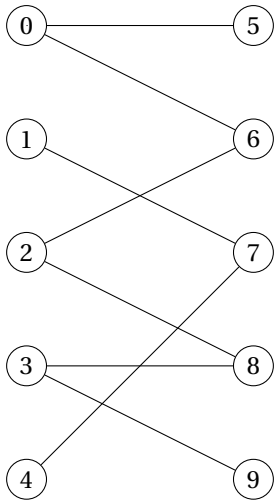

FIGURE 1 – Un exemple de graphe biparti

l'arête (s,t) est un chemin alternant depuis un sommet de S_0. t appartient donc à S_0.
Donc, $s \in S_1 \Rightarrow t \in S_1$. Par symétrie, on a même l'équivalence et donc la propriété recherchée.

Question 17 : Montrer que tout sommet de $X \setminus S_1$ est extrémité d'une arête de C.

Correction :
Soit $x \in X \setminus S_1$. Par définition $x \notin S_0$. On en déduit immédiatement qu'une arête de C a pour extrémité x.

Question 18 : Montrer que le résultat est aussi vrai pour tout sommet de $Y \cap S_1$.

Correction :
Soit $y \in Y \cap S_1$. Par définition de S_1, le sommet y est accessible par un chemin alternant depuis un sommet de S_0. On distingue deux cas :
- Si y est bien extrémité d'une arête de C, la propriété est vérifiée.
- Sinon, y est atteint par un chemin améliorant puisqu'il s'agit d'un chemin alternant dont les deux extrémités ne sont pas incidents à une arête du couplage. Or, le couplage C étant choisi maximal, il n'existe pas de tel chemin.

Question 19 : En déduire que $S_2 = (X \setminus S_1) \cup (Y \cap S_1)$ est une couverture par les sommets de taille $|C|$.

Correction :
Des trois questions précédentes, on déduit que $|S_2| = |C|$. Il reste à montrer qu'il s'agit d'une couverture par les sommets. Soit une arête a de A, on distingue deux cas :
- a appartient à un chemin alternant depuis un sommet de S_0. Auquel cas, elle a une extrémité dans $Y \cap S_1$.
- Sinon, on distingue deux cas :
  - a n'appartient pas à C, auquel cas son extrémité dans X ne peut appartenir à un chemin alternant depuis S_0 puisqu'il serait alors prolongeable par a. Elle a donc une extrémité dans $X \setminus S_1$.
  - Sinon, le seul chemin alternant possible depuis S_0 menant à son extrémité dans X passe par a puisqu'il ne peut avoir deux arêtes de C qui lui soient incidentes. Elle a donc aussi une extrémité dans $X \setminus S_1$.

Ceci conclut la preuve de la propriété recherchée.

Question 20 : En déduire le théorème de König.

Correction :
Le couplage maximal C a pour cardinal $|C|$, et nous venons de montrer qu'il existe une couverture par les sommets S_2 telle que $|S_2| = |C|$. D'après les résultats précédents, le cardinal maximal d'un couplage est inférieur ou égal au cardinal minimal d'une couverture par les sommets. Puisque nous avons trouvé une couverture de

taille égale à celle du couplage maximal, il s'ensuit que le cardinal maximal d'un couplage est égal au cardinal minimal d'une couverture par les sommets. C'est précisément l'énoncé du théorème de König.

3.2 Implémentation

Dans cette partie, nous allons implémenter des fonctions pour exploiter la démonstration du théorème de König dans un graphe biparti. Plus précisément, nous allons implémenter des algorithmes pour calculer un couplage maximum et une couverture par les sommets, puis déduire un ensemble indépendant maximum.

Nous nous appuierons sur la structure de la section précédente pour représenter les graphes bipartis. L'attribut `tableau` de taille n permet de connaître la bipartition du graphe. Un sommet s appartient à X si et seulement si `part[s]` vaut `true`.

On décide de représenter un couplage avec la même structure de donnée. L'attribut `tableau` permettra d'indiquer si un sommet est extrémité d'une arête du couplage. Si une arête (s,t) est présente dans le couplage, `t` sera le seul élément de `voisins[s]` et réciproquement.

Question 21 : Écrire une fonction `graphe creer_couplage(graphe g)` qui, étant donné un graphe g, crée un couplage vide et le renvoie.

Correction :

```
graphe creer_couplage(graphe g)
{
    graphe c;
    c.n = g.n;
    c.voisins = (liste*) malloc(sizeof(liste)*c.n);
    c.tableau = (bool*) malloc(sizeof(bool)*c.n);
    for (int i = 0; i < c.n; c++)
    {
        c.voisins[i]=NULL;
        c.tableau[i]=false;
    }
    return c;
}
```

Question 22 : Écrire une fonction `void ajouter_arete(graphe g, int s, int t)` qui, étant donné un graphe g et une arête (s,t) ajoute celle-ci au graphe. On pourra supposer qu'elle n'est pas déjà présente.

Correction :
J'ai décidé d'implémenter une fonction auxiliaire `ajouter_arc` permettant d'ajouter un arc entre deux sommets donnés.

```
void ajouter_arc(graphe g, int s, int t)
{
    liste* a = (liste*) malloc(sizeof(liste));
    a->val = t;
    a->suivant = g.voisins[s];
    g.voisins[s]=a;
}

void ajouter_arete(graphe g, int s, int t)
{
    ajouter_arc(g,s,t);
    ajouter_arc(g,t,s);
}
```

Question 23 : Écrire une fonction `void retirer_arete(graphe g, int s, int t)` qui, étant donné un graphe g et une arête (s,t) de celui-ci, la retire. On prêtera attention aux fuites mémoires.

Correction :
Comme pour la question précédente, j'ai ici implémenté une fonction `retirer_arc` qui, étant donnés une liste de voisins et un sommet à retirer, renvoie la liste modifiée.

```
liste* retirer_arc(liste* l, int t)
{
    if (l.val = t)
    {
        liste* next = l->suivant;
        free(l);
        return next;
    }
    l->suivant = retirer_arc(l->suivant, t);
    return l;
}

void retirer_arete(graphe g, int s, int t)
{
    g.voisins[s]=retirer_arc(g.voisins[s],t);
    g.voisins[t]=retirer_arc(g.voisins[t],s);
}
```

Question 24 : Rappeler comment calculer un couplage maximum dans un graphe biparti.

Correction :
Pour calculer un couplage maximum dans un graph biparti, on utilise le théorème de Berge : Un couplage est maximum si et seulement si il n'admet pas de chemin améliorant.

Or, étant donné un chemin améliorant, on peut augmenter le cardinal du couplage en retirant les arêtes déjà présentes dans le chemin et en ajoutant les autres. Ainsi, pour calculer un couplage maximum, il suffit de calculer un chemin améliorant, de l'utiliser pour augmenter le cardinal du couplage et de recommencer jusqu'à ne plus trouver de chemin améliorant.

Pour calculer un chemin améliorant, il suffit de réaliser un parcours en profondeur en alternant entre les arêtes présentes et absentes du couplage.

On représente un chemin à l'aide d'un tableau d'entiers de taille n `succ`. À la case i sera présent le sommet suivant i dans le chemin s'il existe et -1 sinon.

Question 25 : Écrire une fonction `bool parcours_profondeur(graphe g, int s, graphe c, int* chemin, bool derniere_arete)` qui, étant donné un graphe g et un sommet s, réalise un parcours en profondeur en n'exploitant que des chemins alternants dans c. Si le parcours obtient un chemin améliorant, la variable `chemin` sera mise à jour pour en rendre compte. On considérera qu'elle fait bien référence à un espace mémoire alloué de bonne dimension. Le paramètre `derniere_arete` vaudra `true` si et seulement si la dernière arête emprunté appartenait à c.

Correction :
La fonction n'est pas si compliquée qu'elle en a l'air. Il faut simplement veiller à bien distinguer les trois cas possibles :

- On a trouvé une sommet qui n'est pas dans le couplage.
- On vient d'utiliser une arête qui n'est pas dans le couplage.
- On vient d'utiliser une arête qui est dans le couplage.

Le tableau `chemin` permet aussi de savoir si on est déjà passé par un sommet. La case s vaudra -1 si l'on a déjà vu le sommet

```
bool parcours_profondeur(graphe g, int s, graphe c, int* chemin, bool
    derniere_arete)
{
    if (chemin[s] != -1)
    {
        return false;
    }
    if (!derniere_arete)
    {
        if (!c.tableau[s])
        {
            return true;
        }
        else
        {
            chemin[s] = t;
            if (parcours_profondeur(g,c.voisins[s],c,chemin,true))
            {
                return true;
            }
            chemin[s] = -1;
            return false;
        }
    }
    liste* l = g.voisins[s];
    while (l!=NULL)
    {
        int t = l->val;
        chemin[s]=l->val;
        if (t!=c.voisins[s]->val && parcours_profondeur(g,l->val,c,
            chemin,false))
        {
            return true;
        }
        l=l->suivant;
    }
    chemin[s]=-1;
    return false;
}
```

Question 26 : Écrire une fonction `bool chemin_ameliorant(graphe g, graphe c, int* chemin)` qui, étant donné un graphe g et un couplage c, teste si celui-ci possède un chemin améliorant. Si c'est bien le cas, `chemin` sera modifié en fonction.

Correction :

```
bool chemin_ameliorant(graphe g, graphe c, int* chemin)
{
    for (int s = 0; s < g.n; s++)
    {
        if (!c.tableau[s] && parcours_profondeur(g,s,c,chemin,true))
        {
            return true;
        }
    }
    return false;
}
```

Question 27 : Écrire une fonction `void ameliorer(graphe c, int* chemin)` qui, étant donné un couplage c et un chemin supposé améliorant, modifie le couplage pour augmenter son cardinal.

Correction :
On parcourt les arêtes du chemin. Pour chaque arête, si elle est déjà présente dans c, on la retire sinon on l'ajoute.

```
void ameliorer(graphe c, int* chemin)
{
    for (int i = 0; i < c.n; i++)
    {
        if (chemin[i]!=-1)
        {
            c.tableau[i]=true;
            int j = chemin[i];
            if (c.voisins[i] = j)
            {
                retirer_arete(c,i,j);
            }
            else
            {
                ajouter_arete(c,i,j);
            }
        }
    }
}
```

Question 28 : Écrire une fonction `graphe calculer_couplage_max(graphe g)` qui prend en entrée un graphe biparti g et renvoie un couplage maximum.

Correction :
On doit ici utiliser les fonctions précédentes en prêtant bien attention à allouer l'espace nécessaire pour les différentes variables utilisées. La fonction `nettoyer` permet ici de réinitialiser les valeurs du tableau `chemin` à -1.

```
void nettoyer(int* chemin, int taille)
{
    for (int i = 0; i < taille; i++)
    {
        chemin[i]=-1;
    }
}

graphe calculer_couplage_max(graphe g)
{
    graphe c = creer_couplage(g);
    int* chemin = (int*) malloc(sizeof(int)*c.n);
    nettoyer(chemin, c.n);
    while (chemin_ameliorant(g,c,chemin))
    {
        ameliorer(c,chemin);
        nettoyer(chemin, c.n);
    }
    return c;
}
```

Question 29 : Écrire une fonction `bool* calculer_couverture_par_sommets(graphe g, graphe c)` qui, étant

donné un graphe biparti g et un couplage maximum c, renvoie une couverture minimale par les sommets, représentée par un tableau de booléens.

Correction :
On implémente une fonction `parcours_alternant` qui réalise un parcours en profondeur en construisant des chemins alternants. Cette fonction permet ainsi de calculer S_1 à partir de S_0. Une fois S_1 calculé, on calcule $(X \setminus S_1) \cup (Y \cap S_1)$.

```
void parcours_alternant(graphe g, int s, graphe c, bool* deja_vus, bool
    derniere_arete)
{
    deja_vus[s]=true;
    if (!derniere_arete)
    {
        int t=c.voisins[s]->val;
        parcours_profondeur(g,t,c,chemin,true)
    }
    else
    {
        for (liste* l = g.voisins[s]; l!=NULL; l=l->suivant)
        {

            int t=l->val;
            if (t!=c.voisins[s]->val)
            {
                parcours_profondeur(g,t,c,chemin,false);
            }
        }
    }
}

bool* calculer_couverture_par_sommets(graphe g, graphe c)
{
    bool* couv = (bool*) malloc(sizeof(bool)*c.n);
    for (int i = 0; i < g.n; i++)
    {
        if (g.tableau[i] && !c.tableau[i])
        {
            parcours_alterant(g,i,c,couv,true);
        }
    }
    for (int i = 0; i < g.n; i++)
    {
        if (g.tableau[i])
        {
            couv[i] = !couv[i];
        }
    }
    return couv;
}
```

Question 30 : Implémenter une fonction `bool* calculer_ensemble_independant_maximum(graphe G)` qui prend en entrée un graphe g, et qui renvoie un ensemble indépendant maximum correspondant.

Correction :
On utilise les fonctions précédentes en libérant la mémoire allouée au cours du processus.

```
bool* calculer_ensemble_independant_maximum(graphe G)
{
    graphe c = calculer_couplage_max(graphe G);
    bool* couv = calculer_couverture_par_sommets(G,c);
    bool* indep = extraire_indep(couv, G.n);
    free(couv);
    free(c.tableau);
    for (int i = 0; i < c.n; i++)
    {
        liste* l = c.voisins[i];
        while(l != NULL)
        {
            liste* next = l->suivant;
            free(l);
            l=next;
        }
    }
    return indep;
}
```

Question 31 : Quel est l'ensemble indépendant obtenu sur le graphe de la figure 1 ?

Correction :
On obtient l'ensemble $\{1,4,5,6,8,9\}$ qui est bien indépendant.

4 Graphes triangulés

4.1 Théorème de Menger

Soit X et Y deux ensembles de sommets d'un graphe G.

- Un XY-séparateur est un ensemble de sommets tel que le graphe obtenu en supprimant les sommets de cet ensemble ne contient aucun chemin de X à Y. On appelle sa taille le nombre de sommets le composant.
- Un XY-connecteur est un sous-graphe de G tel que chacune de ses composantes est un chemin disjoint des autres de X vers Y. Les chemins doivent même être disjoints en leur extrémités. On appelle taille d'un XY-connecteur le nombre de chemins le composant.

On cherche à prouver le théorème suivant appelé théorème de Menger :

> Pour toute paire d'ensemble de sommets X, Y, la taille minimale d'un XY-séparateur est égale à la taille maximale d'un XY-connecteur.

Moins formellement, le nombre minimal de sommets à retirer à G pour déconnecter X de Y est exactement le nombre de chemins disjoints de X à Y.

Question 12 : Prouver que la taille minimale d'un XY-séparateur est supérieure à la taille maximale d'un XY-connecteur.

Correction :
Soit G' un XY-connecteur de taille maximale et S' un XY-séparateur de taille minimale. Lorsque l'on supprime S', il ne reste aucun chemin de X à Y. En particulier, il ne reste aucun chemin dans G'. Puisqu'il sont disjoints, on en déduit que chaque chemin de G' possède au moins un sommet dans S'.

On note s la taille minimale d'un XY-séparateur de G et l'on suppose que G contient une arête (x, y). On note G' le graphe obtenu en la retirant de G. On suppose de plus que G' vérifie la propriété que l'on cherche à démontrer.

Question 13 : Montrer que si S' est un XY-séparateur de G' de taille minimale strictement plus petite que s, $R := S' \cup \{x\}$ et $T := S' \cup \{y\}$ sont des XY-séparateurs de G de taille minimale.

Correction :
Lorsque S' est retiré de G, il existe un chemin d'un sommet de X à Y puisque S' est trop petit pour être un XY-séparateur de G. Ce chemin passe par $\{x, y\}$ puisque si l'on retire cette arête, S' est un XY-séparateur. Si l'on retire R de G, cela revient à retirer plus que S' et $\{x, y\}$ de G. Les ensembles X et Y sont alors bien séparés.

On en déduit que R est un XY-séparateur dans G. Le raisonnement est identique pour T.

Question 14 : Montrer que tout XR-séparateur de G' est un XY-séparateur de G. On admettra qu'il en est de même pour un TY-séparateur de G'.

Correction :
Pour cette question, il faut admettre que x est accessible depuis X dans G'. Si ce n'est pas le cas, on pourrait échanger x et y.
Soit R' un XR-séparateur de G'. On sait que dans G, une fois R' retiré, y n'est plus accessible depuis X. R' est donc aussi un XR-séparateur dans dans G. Puisque chaque chemin de X à Y intersecte R', on en déduit qu'il s'agit aussi d'un XY-séparateur.

Question 15 : Prouver qu'il existe un XY-connecteur de G de taille s.

Correction :
On peut appliquer l'hypothèse selon laquelle G' vérifie la propriété. On en déduit qu'il existe un XR-connecteur de taille s et un TY-connecteur de taille s. Puisque R et T sont aussi de taille s, chaque sommet de R et T sont extrémités d'un chemin disjoint. D'après la questions précédente, retirer R ou T sépare X et Y. On en déduit que les chemins disjoints des deux connecteurs ne peuvent avoir que leurs extrémités en commun. On peut donc construire s chemins disjoints de X à Y en les mettant bout à bout et en passant par $\{x, y\}$ pour celui ayant ces sommets pour extrémités.

Question 16 : Prouver soigneusement par récurrence le théorème recherché.

Correction :
On raisonne par récurrence sur le nombre m d'arêtes dans le graphe.

- Si $m = 0$, pour tout $X, Y \subset S$, pour séparer X et Y, il suffit de retirer les sommets en commun. La taille d'un XY-séparateur minimal est de $|X \cap Y|$. Il s'agit aussi de la taille d'un XY-connecteur maximal le graphe ne présentant aucune arête.
- Supposons la propriété vraie jusqu'au rang m. Soit G un graphe un $m+1$ arêtes et X, Y, deux sous-ensembles de sommets. Soit $\{x, y\}$ une arête du graphe. On considère G' le graphe G dont on a retiré l'arête $\{x, y\}$. On note s la taille minimale d'un XY-séparateur dans G. On distingue deux cas :
  - G' contient un XY-séparateur de taille strictement plus faible que s, on applique le résultat des questions précédentes et on obtient bien que G contient un XY-connecteur de taille s.
  - G' admet un XY-séparateur de taille minimale égale à s. Par hypothèse de récurrence, il admet un XY-connecteur de taille s. C'est aussi un XY-connecteur dans G.

  On a montré dans tous les cas que G contient un XY-connecteur de taille s.

Par récurrence, on a démontré le théorème de Menger quelque soit le nombre d'arêtes du graphe.

4.2 Recherche d'un ensemble indépendant

Un graphe est dit triangulé si tous cycle sans corde possède au plus 3 sommets.
On cherchera dans cette partie, à construire un algorithme polynomial de recherche d'un ensemble indépendant maximum dans de tels graphes.

On dit qu'un sommet s de G est riche si ses voisins sont tous reliés entre eux.

On suppose qu'il existe u et v sont deux sommets non adjacents dans G. On note V_u l'ensemble des voisins de u et V_v l'ensemble des voisins de v. Enfin, on note K un $V_u V_v$-séparateur minimal.

Question 17 : Soit x et y deux sommets de K. Montrer qu'il existe un cycle élémentaire passant par x, y, u et v.

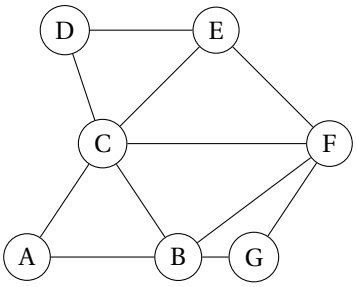

FIGURE 2 – Un graphe triangulé

Correction :
Puisque K est un $V_u V_v$-séparateur minimal, on sait qu'il existe un $V_u V_v$-connecteur de même taille. Exactement un chemin de celui-ci passe par x tandis qu'un autre passe par y. On peut raccourcir ces chemins en conservant les mêmes sommets de passage. Pour cela, on prenant une arrête qui nous permet de sauter des sommets à chaque fois que c'est possible. Ainsi, on obtient deux chemins disjoints de V_u vers V_v qui passent par x et y. En ajoutant u et v pour faire la liaison, on obtient bien un cycle élémentaire passant par x, y, u et v.

Question 18 : En déduire que tous les sommets de K sont voisins entre eux.

Correction :
Il existe un cycle passant par x, y, u et v. Puisque le graphe est triangulé et le cycle de longueur 4, il admet une corde. Deux cordes sont possibles, l'une entre u et v et l'autre entre x et y. Puisque u et v ont été choisis non adjacents, on en déduit qu'elle est nécessairement entre x et y.
Puisque cette propriété est vraie pour tout $x, y \in K$, on en déduit que les sommets de K sont tous voisins entre eux.

Question 19 : On considère C_u les sommets accessibles depuis u sans passer par K. En étudiant $C_u \cup K$, montrer qu'il existe deux sommets riches non adjacents dans G.

Correction :
On raisonne par récurrence sur le nombre de sommets. Si il existe deux sommets non adjacents dans le graphe restreint à $C_u \cup K$, on applique l'hypothèse de récurrence (celui-ci étant bien triangulé) et on en déduit l'existence de deux sommets riches non adjacents. Le cas de base est celui où tous les sommets de $C_u \cup K$ sont reliés entre eux. u est alors riche. On peut alors réaliser le même raisonnement sur $C_v \cup K$ et obtenir ainsi deux sommets riches non adjacents.

On a ainsi prouvé la propriété suivante :

Si un graphe triangulé possède deux sommets non adjacents, il existe deux sommets riches non adjacents.

On utilise ici le type de la seconde partie pour représenter des graphes. L'attribut `tableau` indique si un somme a été retiré du graphe.

Question 20 : Écrire une fonction `bool est_riche(graphe g, int s)` qui teste si un sommet s d'un graphe g est riche. On ne prêtera pas attention aux sommet ayant été retirés.

Correction :

```
bool est_present(liste* l, int s)
{
    for (liste* temp = l; temp !=NULL; temp = temp->suivant)
    {
        if (temp->val == s)
        {
            return true;
        }
    }
    return false;
}

bool est_riche(graphe g, int s)
{
    liste* l = g.voisins[s];
    for (liste* temp = l; temp !=NULL; temp = temp -> suivant)
    {
        if (!g.tableau[temp->val])
        {
            liste* tempbis = temp -> suivant;
            while (tempbis != NULL)
            {
                if (g.tableau[tempbis->val])
                {
                    if (!est_present(g.voisins[tempbis->val], s))
                    {
                        return false;
                    }
                    tempbis=tempbis->suivant;
                }
            }
        }
    }
    return true;
}
```

Question 21 : Écrire une fonction `int trouver_riche(graphe g)` qui, étant donné un graphe, en renvoie un sommet riche. On renverra -1 dans le cas où il n'en existe pas.

Correction :

```
int trouver_riche(graphe g)
{
    for (int s = 0; s < g.n; s++)
    {
        if (!g.tableau[s] && est_riche(g,s))
        {
            return s;
        }
    }
}
```

On propose d'utiliser l'algorithme glouton en sélectionnant à chaque itération un sommet riche.

Question 22 : Détailler l'exécution de l'algorithme sur le graphe de la figure 2.

Algorithme 2 Glouton_triangule(G)

$I \leftarrow \emptyset$
tant que $G \neq \emptyset$ **faire**
  $s \leftarrow$ un sommet riche de G.
  $I \leftarrow I \cup \{s\}$.
  pour t voisins de s **faire**
    Retirer t de S.
  fin pour
  Retirer s de S.
fin tant que
return I

Correction :
On décompose l'exécution de l'algorithme sur l'exemple fourni. Ici, on a choisi A, D puis F qui étaient bien des sommets riches à chaque fois.

I	S	s
$\emptyset$	$\{A,B,C,D,E,F,G\}$	
$\{A\}$	$\{D,E,F,G\}$	A
$\{A,D\}$	$\{E,F,G\}$	D
$\{A,D,F\}$	$\{\}$	F

Question 23 : Montrer que la propriété "I peut être complété en un ensemble indépendant maximum" est un invariant de la boucle Tant que.

Correction :
Le raisonnement est similaire à la preuve de correction de l'algorithme de Kruskal.
On commence par remarquer que la propriété est vraie avant la première itération.
Supposons la propriété vraie au début d'un itération. Il existe I_1 un ensemble indépendant maximum tel que $I \subset I_1$. On distingue deux cas :

- $s \in I_1$. Dans ce cas, puisque l'on ajoute s à I, il est toujours inclus dans un ensemble indépendant maximum puisqu'il est inclus dans I_1.
- $s \notin I_1$.Dans ce cas, on sait qu'un des voisins s' de s est dans I_1 sinon il ne serait pas maximum. Puisque tous les voisins de s sont aussi voisins de s', on en déduit que $I_1 \setminus \{s'\} \cup \{s\}$ est aussi indépendant. C'est même un indépendant maximum puisque de même taille que I_1. $I \cup \{s\}$ est bien inclus dans un ensemble indépendant de cardinal maximum.

Question 24 : Prouver que l'algorithme termine et renvoie bien un ensemble indépendant maximum.

Correction :
La taille de S est un variant de boucle. On a aussi identifié un invariant de boucle. Celui-ci est vrai après la dernière itération. L'ensemble I obtenu après la dernière itération peut être complété en un ensemble indépendant maximum. Or, tous les sommets de S ont été retirés car ils étaient ajoutés à I ou voisins d'un sommet de I. Aucun sommet de S ne peut donc être ajouté sans perdre la propriété d'indépendance de I. Il est donc lui même maximum.

Question 25 : Écrire une fonction `bool* independant_triangule (graphe g)` qui étant donné un graphe g triangulé, renvoie un ensemble indépendant maximum.

Correction :

```
bool* independant_triangule(graphe g)
{
    bool* res = (bool*) malloc(sizeof(bool)*g.n);
    for (int i = 0; i < g.n; i++)
    {
        res[i] = false;
    }
    int restants = g.n;
    while (restants > 0)
    {
        int s = trouver_riche(g);
        for (liste* temp = g.voisins[s]; temp != NULL; temp = temp->
            suivant)
        {
            if (!g.tableau[temp->val])
            {
                restants = restants-1;
                g.tableau[temp->val] = false;
            }
        }
        g.tableau[s] = false;
        res[s] = true;
    }
    return res;
}
```

Question 26 : Analyser la complexité de l'algorithme.

Correction :
Tester si un sommet est riche est ici réalisé en $O(n^3)$. Pour trouver un sommet riche, on réalise ce test au plus n fois. Enfin, on retire au moins un sommet de G à chaque itération de l'algorithme. On en déduit donc que l'algorithme s'exécute en $O(n^5)$.